DETECTING AND DEFENDING MACHINE LEARNING-BASED INTRUSION DETECTION SYSTEMS AGAINST ADVERSARIAL EVASION AND DATA-POISONING ATTACKS

A PROJECT REPORT

Submitted by

SANDEEP S **2116231901045**

THENMOZHI R **2116231901064**

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "**DETECTING AND DEFENDING MACHINE LEARNING-BASED INTRUSION DETECTION SYSTEMS AGAINST ADVERSARIAL EVASION AND DATA-POISONING ATTACKS**" is the bonafide work of "**SANDEEP S. (2116231901045), THENMOZHI R. (2116231901064)**" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

6/10/26

SIGNATURE

Dr. P. Tamilselvi, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

Professor

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

6/10/26

SIGNATURE

Ms. Swathi Lakshmi S, M.Tech.,

SUPERVISOR

Assistant Professor

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 06/10/2026

Internal Examiner

6/10/26

External Examiner

ABSTRACT

Machine learning (ML)-based Network Intrusion Detection Systems (NIDS) are widely used to detect malicious network activity, but they are vulnerable to adversarial manipulation, where attackers evade detection by modifying input traffic at inference time or degrade model reliability by poisoning training data. This paper presents AdversaGuard, a unified framework for evaluating and improving the robustness of ML-based NIDS against both evasion and data-poisoning attacks. The framework evaluates Random Forest, XGBoost, and Deep Neural Network (DNN) models on the CICIDS2017 and NSL-KDD datasets, simulating evasion attacks (FGSM and PGD) and poisoning attacks (label-flipping and backdoor) in a controlled setting. A layered defence strategy combining data sanitisation, adversarial training, feature squeezing, and autoencoder-based runtime detection is applied, and individual and combined defences are compared. AdversaGuard further integrates SHAP-based Explainable AI to identify influential features, a configurable risk-scoring module that generates Low, Medium, High, or Critical alerts, an interactive attack simulator, automated security reporting, and a Streamlit dashboard for interactive monitoring. Performance is measured before, during, and after attacks using accuracy, precision, recall, F1-score, attack success rate, detection rate, and false-positive rate. Rather than claiming guaranteed protection against future attacks, AdversaGuard provides a practical, interpretable, and systematic framework for testing, comparing, and strengthening the adversarial robustness of ML-based intrusion detection systems.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Ms. Swathi Lakshmi S, Assistant Professor,** for her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

SANDEEP S 2116231901045

THENMOZHI R 2116231901064

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	**ii**
	LIST OF TABLES	**vi**
	LIST OF FIGURES	**vii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	3
	1.3 OBJECTIVES	4
	1.4 SCOPE OF THE PROJECT	8
	1.5 SECURITY REQUIREMENTS AND CONSIDERATIONS	11
	1.6 SDG ALIGNMENT	13
2	**LITERATURE REVIEW**	**14**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	14
	2.2 RESEARCH GAPS	14
	2.3 PROPOSED SYSTEM	15
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	16
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	16

	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	17
3	**SYSTEM DESIGN**	**18**
	3.1 SYSTEM ARCHITECTURE	18
	3.2 MODULE DESIGN	19
	3.3 DATA FLOW DIAGRAM	20
	3.4 USE CASE DIAGRAM	21
	3.5 ACTIVITY DIAGRAM	22
	3.6 SEQUENCE DIAGRAM	23
	3.7 THREAT MODEL AND ATTACK SURFACE	24
	3.8 SECURITY CONTROLS	24
4	**SYSTEM DESCRIPTION**	**25**
	4.1 DATA COLLECTION	25
	4.2 DATA PREPROCESSING	25
	4.2.1 ALGORITHMS / METHODOLOGY	26
	4.2.2 IMPLEMENTATION TECHNOLOGY	27
	4.2.3 RESULTS AND DISCUSSION	27
5	**CONCLUSION**	**29**
	REFERENCES	**30**

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
1.1	RISK SCORE LEVEL	7
1.2	SDG ALIGNMENT	13
4.1	IMPLEMENTATION TECHNOLOGIES	27

LIST OF FIGURES

FIGURE	TITLE	PAGE NO
2.1	GENERAL WORKFLOW OF PROPOSED SYSTEM	15
3.1	ADVERSAGUARD SYSTEM ARCHITECTURE	18
3.2	DATA FLOW DIAGRAM	20
3.3	USE CASE DIAGRAM	21
3.4	ACTIVITY DIAGRAM	22
3.5	SEQUENCE DIAGRAM	23
3.6	ADVERSAGUARD ATTACK SURFACE	24

CHAPTER 1

INTRODUCTION

1.1 Background

Modern computer networks have become an important part of cloud services, enterprise applications, Internet of Things (IoT) devices, web applications, and other digital services. As the amount of network communication increases, the number and complexity of cyberattacks also increase. Attackers continuously develop new techniques to bypass traditional security mechanisms and gain unauthorized access to systems and networks. Therefore, effective monitoring and detection of malicious network traffic is an important requirement for network security.

A Network Intrusion Detection System (NIDS) is a security mechanism that monitors network traffic and identifies suspicious or malicious activities. Traditional intrusion detection systems mainly depend on predefined signatures, rules, thresholds, and manually created patterns. These approaches can detect previously known attacks effectively, but their ability to identify new, modified, or previously unseen attacks is limited.

Machine Learning (ML) has become an important approach for improving intrusion detection. Instead of depending only on predefined rules, ML-based Network Intrusion Detection Systems can learn patterns from previously collected network traffic and classify traffic as normal or malicious. Different machine learning algorithms can be used for this purpose. Random Forest (RF) uses multiple decision trees to perform classification, XGBoost uses gradient-boosted decision trees, and Deep Neural Networks (DNNs) can learn complex non-linear relationships in network traffic data.

The performance of ML-based NIDS is commonly evaluated using network security datasets. In this project, **CICIDS2017** is used as the primary dataset because it contains realistic network traffic and multiple attack categories. **NSL-KDD** is used as a complementary benchmark dataset for additional evaluation.

Although machine learning improves detection capability, it also introduces new security challenges. An ML-based NIDS can be manipulated through adversarial machine learning attacks. These attacks can target either the trained model during prediction or the training data used to build the model.

Two major attack categories are considered in AdversaGuard:

1. **Evasion Attacks** – These attacks modify input samples after the model has been trained so that malicious traffic may be incorrectly classified as normal traffic. The project uses **Fast Gradient Sign Method (FGSM)** and **Projected Gradient Descent (PGD)** for controlled evasion experiments.
2. **Poisoning Attacks** – These attacks manipulate training data before or during model training. The project considers **label-flipping attacks** and **backdoor attacks** to study the effect of corrupted training information on the trained NIDS.

Therefore, evaluating only the accuracy of a model on clean data is not sufficient. A model may provide high accuracy on normal test data while becoming less reliable when its inputs or training data are deliberately manipulated. AdversaGuard addresses this problem by providing an integrated framework for evaluating ML-based NIDS under both clean and adversarial conditions.

The proposed framework also includes multiple defence mechanisms such as **data sanitisation, adversarial training, feature squeezing, autoencoder-based runtime detection, and a combined defence approach**. These mechanisms are evaluated to understand their ability to reduce the impact of adversarial attacks.

To improve the interpretability of model decisions, **Explainable Artificial Intelligence (XAI)** is incorporated using SHAP. SHAP helps identify the contribution of individual network traffic features to a model prediction. The system also includes a configurable **0–100 risk score**, security alerts, an interactive attack simulator, defence comparison, a Streamlit dashboard, and automated security reporting.

The overall workflow of AdversaGuard is:

Dataset → Preprocessing → ML Model → Clean Evaluation → Adversarial Attack → Defence → Runtime Detection → Final Prediction → SHAP Explanation → Risk Assessment → Security Alert → Dashboard → Automated Report

The main purpose of AdversaGuard is not to claim complete protection against all cyberattacks. Instead, it provides a controlled environment for studying the robustness of ML-based NIDS, comparing attack and defence techniques, and understanding how adversarial manipulation affects intrusion detection performance.

1.2 Problem Statement

Machine Learning-based Network Intrusion Detection Systems are generally evaluated using clean datasets and conventional performance metrics such as accuracy, precision, recall, and F1-score. However, high performance on clean network traffic does not necessarily indicate that the model is secure against adversarial manipulation.

Attackers may manipulate network traffic features during prediction to evade detection. Similarly, malicious or incorrect samples may be introduced into the training data to influence the behaviour of the trained model. Evasion attacks such as FGSM and PGD can modify input samples, while poisoning techniques such as label flipping and backdoor attacks can affect the training process.

Many individual approaches focus on a specific ML model, attack type, or defence mechanism. This makes it difficult to obtain a common evaluation of different models and multiple attack and defence techniques. In addition, model predictions may not clearly explain why a particular network flow was classified as malicious or normal.

Therefore, there is a need for an integrated framework that can:

- Evaluate multiple ML-based NIDS models.
- Test models using both clean and adversarial network traffic.
- Simulate evasion and poisoning attacks.
- Apply and compare multiple defence mechanisms.
- Detect suspicious inputs during runtime.
- Explain model predictions using XAI.

- Assign a configurable risk score to detected events.
- Generate security alerts.
- Provide an interactive environment for controlled attack simulation.
- Compare the performance of different defence mechanisms.
- Generate automated security reports.

AdversaGuard is proposed to address these requirements by providing a unified framework for evaluating and improving the robustness of ML-based Network Intrusion Detection Systems against adversarial attacks.

1.3 Objectives

The main objective of AdversaGuard is to develop an integrated framework for evaluating and defending machine learning-based Network Intrusion Detection Systems against adversarial evasion and data-poisoning attacks.

1.3.1 Development of ML-Based NIDS

To develop machine learning-based intrusion detection models using:

- Random Forest (RF)
- XGBoost
- Deep Neural Network (DNN)

These models are used to classify network traffic and provide a common environment for robustness evaluation.

1.3.2 Dataset-Based Evaluation

To evaluate the developed models using:

- CICIDS2017 as the primary dataset.
- NSL-KDD as a complementary benchmark dataset.

1.3.3 Data Preprocessing

To preprocess the network traffic datasets by handling:

- Missing values

- Duplicate records
- Infinite values
- Categorical features
- Feature selection
- Feature scaling
- Data normalization
- Training and testing separation

1.3.4 Clean Performance Evaluation

To establish baseline model performance using clean network traffic before applying any adversarial attack.

The evaluation includes metrics such as:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion matrix

1.3.5 Evasion Attack Simulation

To simulate inference-time adversarial attacks using:

- Fast Gradient Sign Method (FGSM)
- Projected Gradient Descent (PGD)

These attacks are used to study how modifications to network traffic features can affect model predictions.

1.3.6 Poisoning Attack Simulation

To simulate training-time poisoning attacks using:

- Label flipping
- Backdoor attacks

The purpose is to study how manipulated training data can influence the behaviour of the ML-based NIDS.

1.3.7 Adversarial Robustness Evaluation

To compare model performance under:

- Clean conditions
- FGSM attacks
- PGD attacks
- Label-flipping attacks
- Backdoor attacks

This provides a better understanding of the difference between normal model performance and adversarial robustness.

1.3.8 Defence Mechanism Implementation

To implement and evaluate multiple defence techniques:

- Data sanitisation
- Adversarial training
- Feature squeezing
- Autoencoder-based runtime detection
- Combined defence

1.3.9 Runtime Detection

To identify suspicious or adversarial input samples during the prediction stage using an autoencoder-based runtime detection mechanism.

1.3.10 Explainable AI

To integrate SHAP-based Explainable AI to identify important features contributing to model predictions.This helps users understand why a particular network traffic sample was classified as normal or malicious.

1.3.11 Risk Assessment

To calculate a configurable risk score between **0 and 100** based on model prediction, attack information, runtime detection results, and other security indicators.

Table 1.1: Risk Score Level

Risk Score	Risk Level
0-30	Low
31–60	Medium
61–80	High
81–100	Critical

The project uses the following configurable levels:These thresholds are defined as part of the project and are not intended to represent a universal cybersecurity standard.

1.3.12 Security Alert Generation

To generate security alerts when suspicious or high-risk events are identified. Alerts contain important information such as attack type, prediction, risk score, and defence status.

1.3.13 Interactive Attack Simulation

To develop an attack simulator that allows the user to select:

- Dataset
- ML model
- Attack type
- Attack strength
- Number of samples
- Poisoning percentage where applicable
- Defence mechanism

The simulator provides a controlled environment for testing model robustness.

1.3.14 Interactive Dashboard

To develop a Streamlit-based dashboard for:

- Dataset selection
- Model selection

- Model training
- Clean evaluation
- Attack simulation
- Defence selection
- SHAP analysis
- Risk assessment
- Alert monitoring
- Defence comparison
- Report generation

1.3.16 Automated Security Reporting

To generate security reports containing experiment configuration, model performance, attack results, defence performance, risk information, alerts, and explainability results.

1.4 Scope of the Project

1.4.1 Dataset Scope

The project uses two network intrusion datasets:

- **CICIDS2017** – primary dataset.
- **NSL-KDD** – complementary benchmark dataset.

The datasets provide network traffic features and corresponding labels required for ML-based intrusion detection.

1.4.2 Machine Learning Scope

The project focuses on three ML models:

1. Random Forest
2. XGBoost
3. Deep Neural Network

These models are evaluated under clean and adversarial conditions.

1.4.3 Data Preprocessing Scope

The preprocessing stage includes:

- Missing-value handling
- Duplicate removal
- Infinite-value handling
- Categorical feature encoding
- Feature selection
- Feature scaling
- Normalization
- Training and testing split

Proper preprocessing is performed before model training and evaluation.

1.4.4 Adversarial Attack Scope

The project focuses on four attack techniques:

Evasion attacks:

- FGSM
- PGD

Poisoning attacks:

- Label flipping
- Backdoor

The attacks are implemented in a controlled experimental environment.

1.4.5 Defence Scope

The following defence mechanisms are considered:

- Data sanitisation
- Adversarial training
- Feature squeezing
- Autoencoder-based runtime detection
- Combined defence

Their effectiveness is compared using common evaluation metrics.

1.4.6 Explainability Scope

SHAP is used to provide feature-level explanations of ML predictions.

The explanation module helps identify which network traffic features have the greatest contribution to a prediction.

1.4.7 Risk Assessment Scope

The system generates a configurable risk score from 0 to 100 and maps it to Low, Medium, High, or Critical risk levels.

The score is intended to support project-level security analysis and alert prioritisation.

1.4.8 Dashboard Scope

The Streamlit dashboard provides an interactive interface for:

- Dataset selection
- Model selection
- Training
- Clean evaluation
- Attack simulation
- Defence evaluation
- SHAP explanation
- Risk analysis
- Security alerts
- Defence comparison
- Report generation

1.4.9 Reporting Scope

The automated report can include:

- Dataset information
- Selected ML model
- Attack configuration
- Defence configuration
- Evaluation metrics

- Confusion matrix
- Risk score
- Risk level
- Security alerts
- SHAP explanations
- Defence comparison
- Experiment summary

1.4.10 Project Limitations

The project is primarily designed for controlled experimental evaluation using benchmark datasets.

It is not intended to directly replace a production NIDS without additional:

- Real-time network integration
- Security hardening
- Deployment testing
- Performance validation
- Continuous model monitoring
  Evaluation on real organizational traffic

1.5 Security Requirements and Considerations

1.5.1 Adversarial Robustness

The system should evaluate whether ML-based NIDS models maintain acceptable performance when exposed to adversarially manipulated inputs or training data. The system should test evasion and poisoning attacks and compare model performance before and after attacks.

1.5.2 Training Data and Input Security

Training and input data should be validated before being processed by the system. Suspicious, corrupted, or potentially poisoned samples should be identified and removed or isolated where applicable. Proper separation of training and testing data should also be maintained to prevent data leakage.

1.5.3 Secure Attack Simulation and Runtime Security

Attack simulation should be performed only within the controlled project environment using authorised experimental datasets. During runtime, incoming samples should be examined using the runtime detection mechanism to identify suspicious or adversarial inputs.

1.5.4 Model and Defence Integrity

Trained models should be stored and managed securely to prevent unauthorized modification or replacement. Defence mechanisms such as adversarial training, feature squeezing, data sanitisation, and runtime detection should be evaluated to determine their effectiveness against adversarial attacks.

1.5.5 Explainability, Risk Assessment and Alerts

SHAP should provide understandable explanations of important features contributing to model decisions. The risk module should convert security indicators into a configurable risk score, while the alert mechanism should identify and highlight suspicious or high-risk events for further analysis.

1.6 SDG ALIGNMENT

Table 1.1: SBG ALIGNMENT

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
01	SDG 9 – Industry, Innovation and Infrastructure	9.5 – Enhance scientific research and technological capabilities	AdversaGuard applies machine learning and adversarial security techniques to evaluate and improve the robustness of ML-based Network Intrusion Detection Systems.
02	SDG 16 – Peace, Justice and Strong Institutions	16.1 – Reduce violence and related threats	AdversaGuard contributes to safer digital environments by detecting malicious network activities, evaluating security risks, generating alerts, and supporting security monitoring.
03	SDG 4 – Quality Education	4.4 – Develop relevant technical and vocational skills	The project provides practical learning in cybersecurity, machine learning, adversarial attacks, intrusion detection, and security defence techniques.

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Network Intrusion Detection Systems (NIDS) are widely used to monitor network traffic and identify malicious activities. Traditional NIDS mainly use signature-based and anomaly-based approaches. Signature-based systems identify attacks by comparing network traffic with previously known attack patterns, while anomaly-based systems identify unusual behaviour that differs from normal network activity.

With the development of Artificial Intelligence, Machine Learning-based NIDS have become an important security approach. Machine learning models such as Random Forest, XGBoost, and Deep Neural Networks (DNN) can learn patterns from network traffic and classify activities as normal or malicious. These approaches can detect different types of attacks without depending completely on predefined signatures.

However, ML-based NIDS also introduce new security challenges. Attackers can manipulate network inputs to evade detection or modify training data to affect the behaviour of the trained model. Evasion attacks such as FGSM and PGD can modify input features, while poisoning attacks such as label flipping and backdoor attacks can affect the training process.

Existing security approaches include data preprocessing, anomaly detection, adversarial training, data sanitisation, and model-based detection techniques. However, many existing systems focus mainly on detection accuracy and do not provide a complete framework for evaluating multiple adversarial attacks, applying multiple defences, and analysing the security results.

2.2 RESEARCH GAPS

The study of existing ML-based NIDS approaches identifies several important research gaps:

- **Adversarial Vulnerability:** ML-based NIDS models can be vulnerable to adversarial manipulation, which may reduce their detection performance.
- **Limited Attack Evaluation:** Many approaches focus on specific attack types instead of evaluating both evasion and data-poisoning attacks in the same framework.
- **Limited Defence Comparison:** Different defence techniques are often studied separately, making it difficult to determine which defence is more suitable for a particular attack.

- **Lack of Explainability: Traditional detection systems may provide a prediction without clearly explaining why the model classified network traffic as malicious or normal.**
- **Limited Risk Analysis:** Detection results do not always provide a simple risk score or security alert that can help analysts understand the severity of an attack.
- **Lack of Integrated Analysis:** There is a need for a unified framework that combines attack simulation, defence evaluation, explainability, risk assessment, and automated reporting.

2.3 PROPOSED SYSTEM

To address the identified research gaps, AdversaGuard is proposed as an integrated framework for evaluating and improving the robustness of Machine Learning-based Network Intrusion Detection Systems against adversarial attacks.The system uses Random Forest, XGBoost, and Deep Neural Network (DNN) models to detect normal and malicious network traffic. The trained models are evaluated using different adversarial attacks, including FGSM, PGD, Label Flipping, and Backdoor attacks.

AdversaGuard also applies different defence techniques such as Data Sanitisation, Adversarial Training, Feature Squeezing, and Autoencoder-based Runtime Detection. The framework further integrates Explainable AI using SHAP, risk scoring, security alerts, attack simulation, defence comparison, and automated security reporting. The complete system is presented through a Streamlit dashboard, allowing a security analyst to configure experiments, simulate attacks, analyse results, compare defence techniques, and generate security reports.

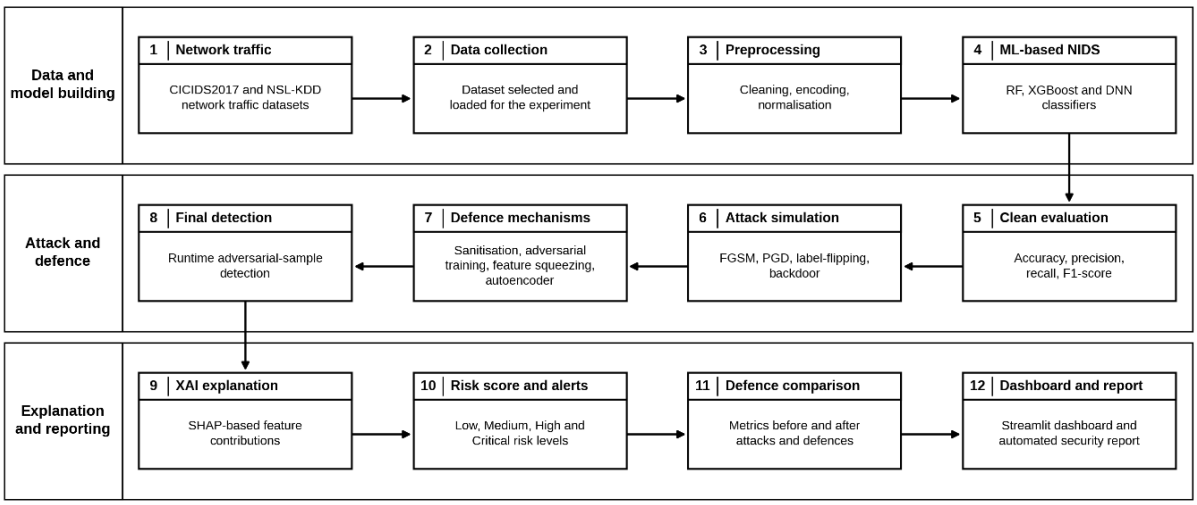


Figure 2.1: General Workflow of Proposed System

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed AdversaGuard system begins by collecting network traffic datasets and performing data preprocessing to clean and prepare the data for machine learning. The processed data is then used to train Random Forest, XGBoost, and DNN models for intrusion detection. After model training, the system evaluates the models using both clean network traffic and adversarially manipulated data. Evasion attacks such as FGSM and PGD are used to test whether malicious traffic can bypass the trained models. Poisoning attacks such as label flipping and backdoor attacks are used to analyse the effect of manipulated training data.
Different defence mechanisms are then applied to improve model robustness. The performance of the models before and after applying defence techniques is compared using suitable evaluation measures.

The system also uses SHAP-based Explainable AI to identify important features influencing model predictions. A risk scoring and alert mechanism provides a simple indication of the security severity. Finally, the Streamlit dashboard presents the results and generates automated security reports for further analysis.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

The major objectives of the proposed AdversaGuard system are:

- **Develop an ML-Based NIDS** using Random Forest, XGBoost, and DNN for detecting normal and malicious network traffic.
- **Evaluate Evasion Attacks** such as FGSM and PGD to identify vulnerabilities in trained intrusion detection models.
- **Evaluate Poisoning Attacks** such as label flipping and backdoor attacks to analyse their impact on model behaviour.
- **Apply Defence Techniques** including data sanitisation, adversarial training, feature squeezing, and autoencoder-based runtime detection.
- **Provide Explainable Security Analysis** using SHAP to understand the features responsible for model predictions.
- **Implement Risk Scores and Alerts** to provide a simple and understandable representation of detected security threats.
- **Provide Attack Simulation and Defence Comparison** so that different attacks and defence mechanisms can be evaluated systematically.

- **Generate Automated Security Reports** containing important experimental results and analysis through the Streamlit dashboard.

2.3.3 ADVANTAGES OF PROPOSED SYSTEM

The proposed AdversaGuard system provides several advantages:

- Improved Model Robustness: Helps evaluate and improve the robustness of ML-based NIDS against different adversarial attacks.
- Multiple Attack Evaluation: Supports both evasion attacks and data-poisoning attacks, providing broader security testing.
- Multiple Defence Techniques: Allows different defence mechanisms to be applied and compared under controlled experimental conditions.
- Explainable AI: SHAP provides an understandable explanation of the important features influencing the model's prediction.
- Risk Assessment: Risk scores and security alerts help security analysts identify and prioritise potentially serious threats.
- Interactive Attack Simulation: The dashboard allows analysts to simulate different adversarial attacks and observe their effect on the NIDS models.
- Defence Comparison: Enables comparison of different defence techniques to understand their relative effectiveness.
- Automated Reporting: Generates security reports containing attack results, defence performance, explanations, and risk information.
- Centralised Dashboard: Streamlit provides a single interface for configuring experiments, running simulations, viewing results, and analysing security performance.

CHAPTER 3

SYSTEM DESIGN

3.1 System Architecture

AdversaGuard is designed as an integrated framework for testing and defending ML-based Network Intrusion Detection Systems against adversarial attacks. It combines dataset preprocessing, ML models, attack simulation, defence mechanisms, runtime detection, Explainable AI, risk assessment, alerts, and reporting.

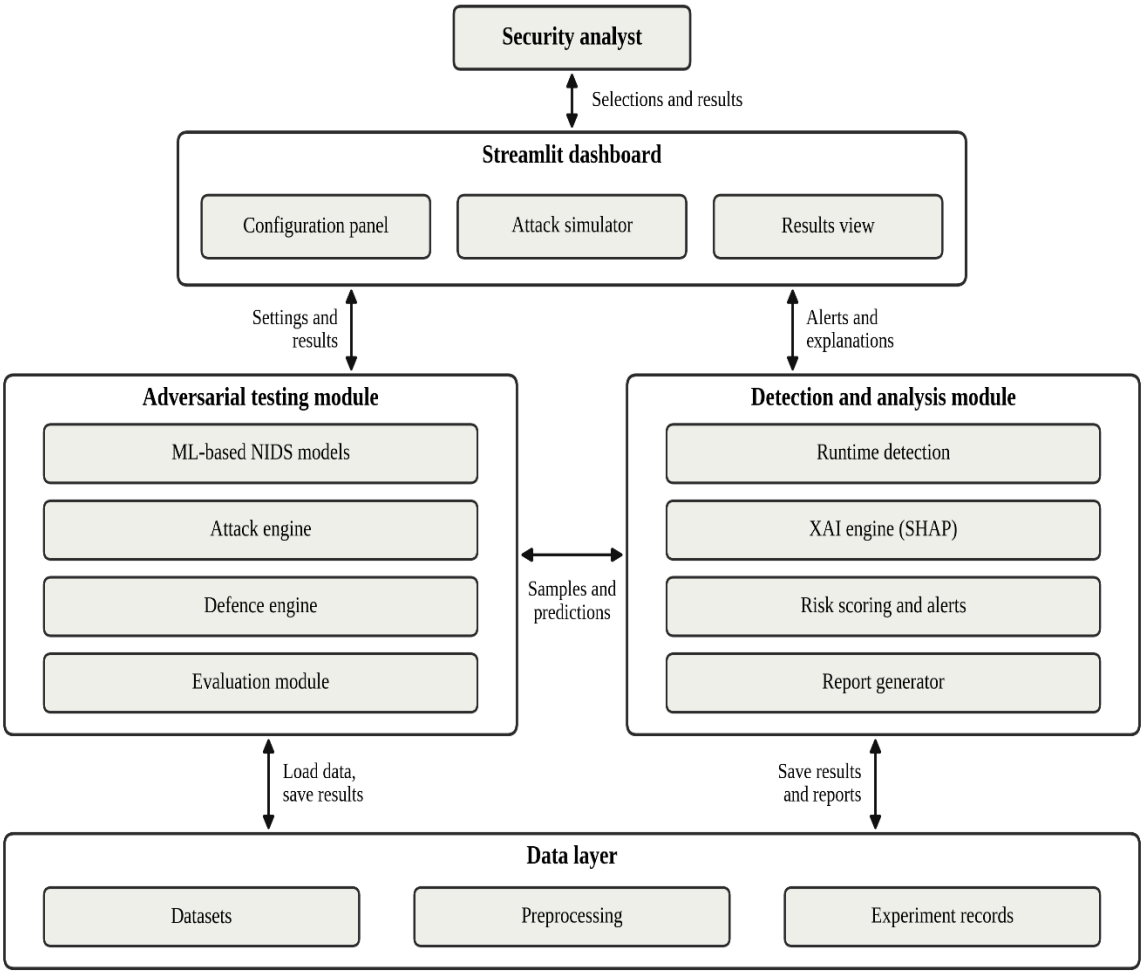


Figure 3.1: AdversaGuard System Architecture

3.2 Module Design

AdversaGuard consists of the following main modules:

3.2.1 Dataset Module

Manages CICIDS2017 and NSL-KDD datasets and prepares them for processing.

3.2.2 Preprocessing Module

Cleans the data by handling missing values, duplicates, invalid values, encoding, scaling, and feature selection.

3.2.3 Machine Learning Module

Supports three models:

- Random Forest
- XGBoost
- Deep Neural Network

3.2.4 Attack Simulation Module

Tests model robustness using:

- FGSM
- PGD
- Label Flipping
- Backdoor

3.2.5 Defence Module

Provides:

- Data Sanitisation
- Adversarial Training
- Feature Squeezing
- Autoencoder Runtime Detection
- Combined Defence

3.2.6 Explainability and Risk Module

Uses SHAP to explain predictions and generates a configurable 0–100 risk score.

3.2.7 Dashboard and Reporting Module

The Streamlit dashboard displays predictions, explanations, risk levels, alerts, and defence comparisons. It also supports automated report generation.

3.3 Data Flow Diagram

The DFD shows how data moves through the AdversaGuard system.

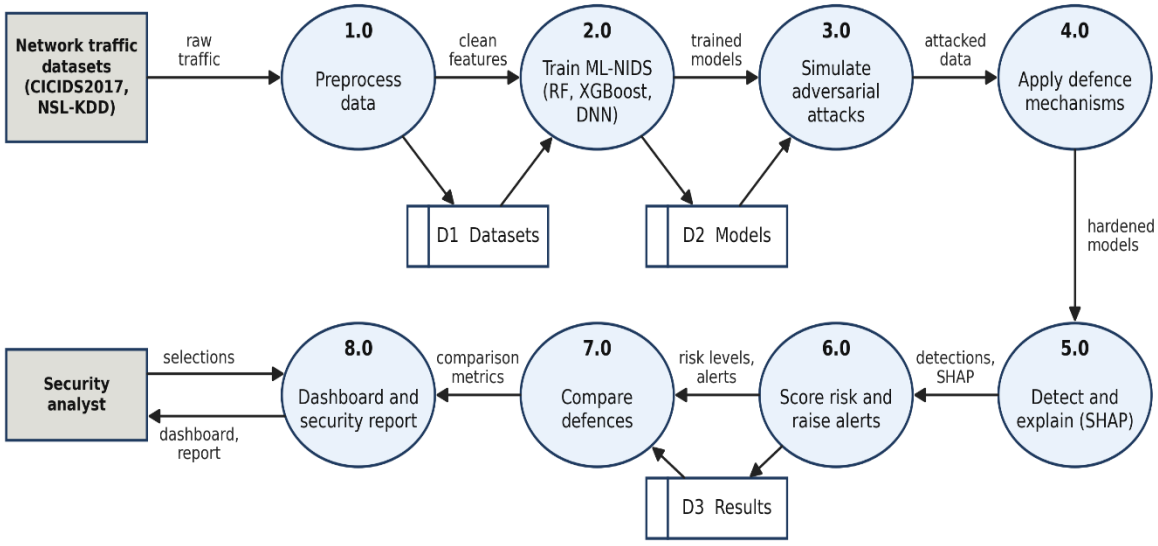


Figure 3.2: Data Flow Diagram

3.4 Use Case Diagram

The main actor is the **Security Analyst / Researcher**.

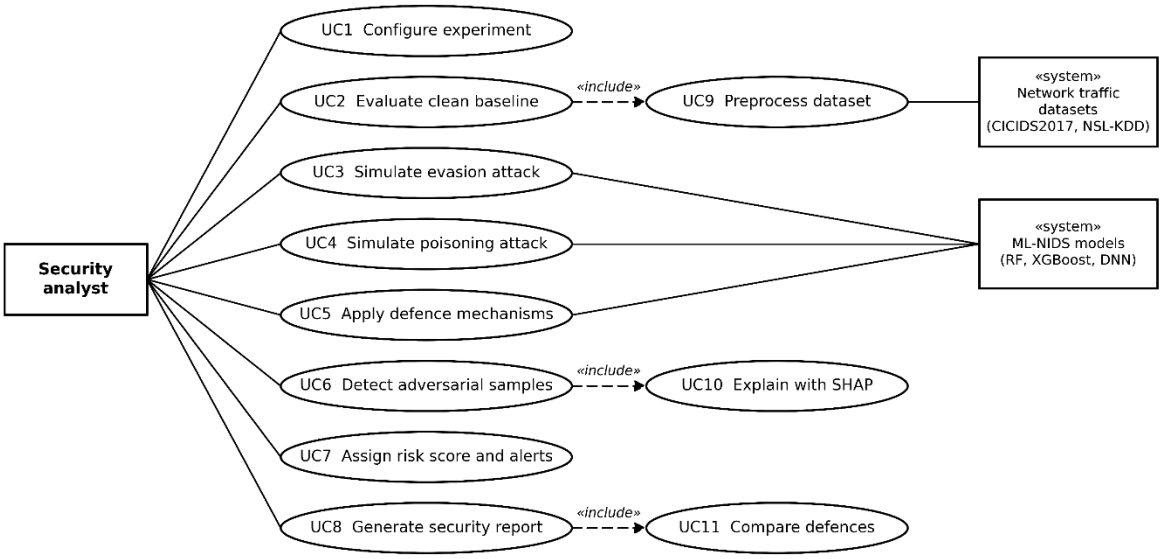


Figure 3.3: Use Case Diagram

The user can:

- Select dataset
- Select ML model
- Train and evaluate model
- Select and run attacks
- Select defence
- Compare defences
- View SHAP explanation
- View risk score
- View alerts
- Generate report

An administrator can additionally manage datasets, experiment records, and reports.

3.5 Activity Diagram

The activity diagram represents the complete workflow of the system.

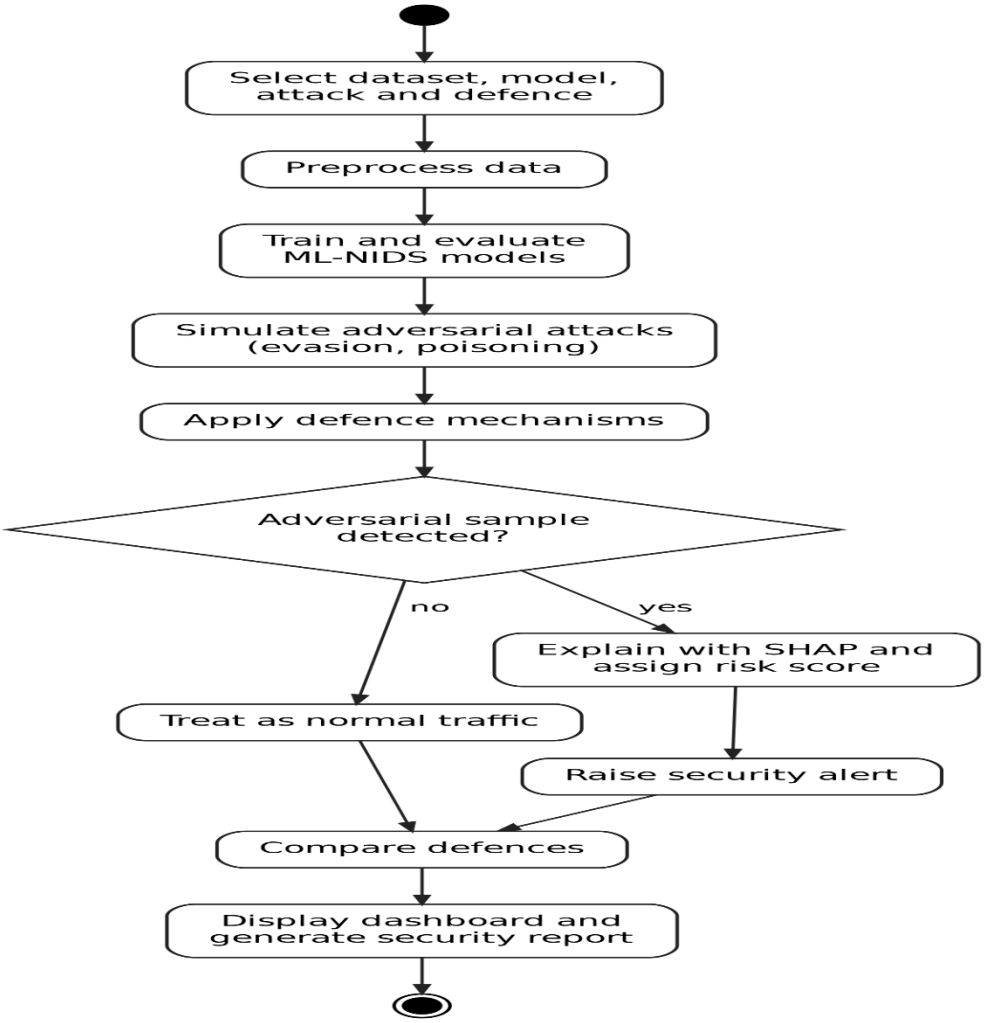


Figure 3.4: Activity Diagram

The activity diagram outlines the AdversaGuard workflow, from dataset, model, attack and defence selection through preprocessing, model evaluation, adversarial attack simulation and defence application. Detected adversarial samples are explained with SHAP, assigned a risk score and raised as security alerts, while undetected traffic is treated as normal. The workflow concludes with defence comparison, dashboard display and automated report generation.

3.6 Sequence Diagram

The sequence diagram shows communication between the main system components.

Use these entities:

User → Dashboard → Dataset → Preprocessing → ML Model → Attack Simulator → Defence → Runtime Detector → SHAP → Risk → Alert → Report

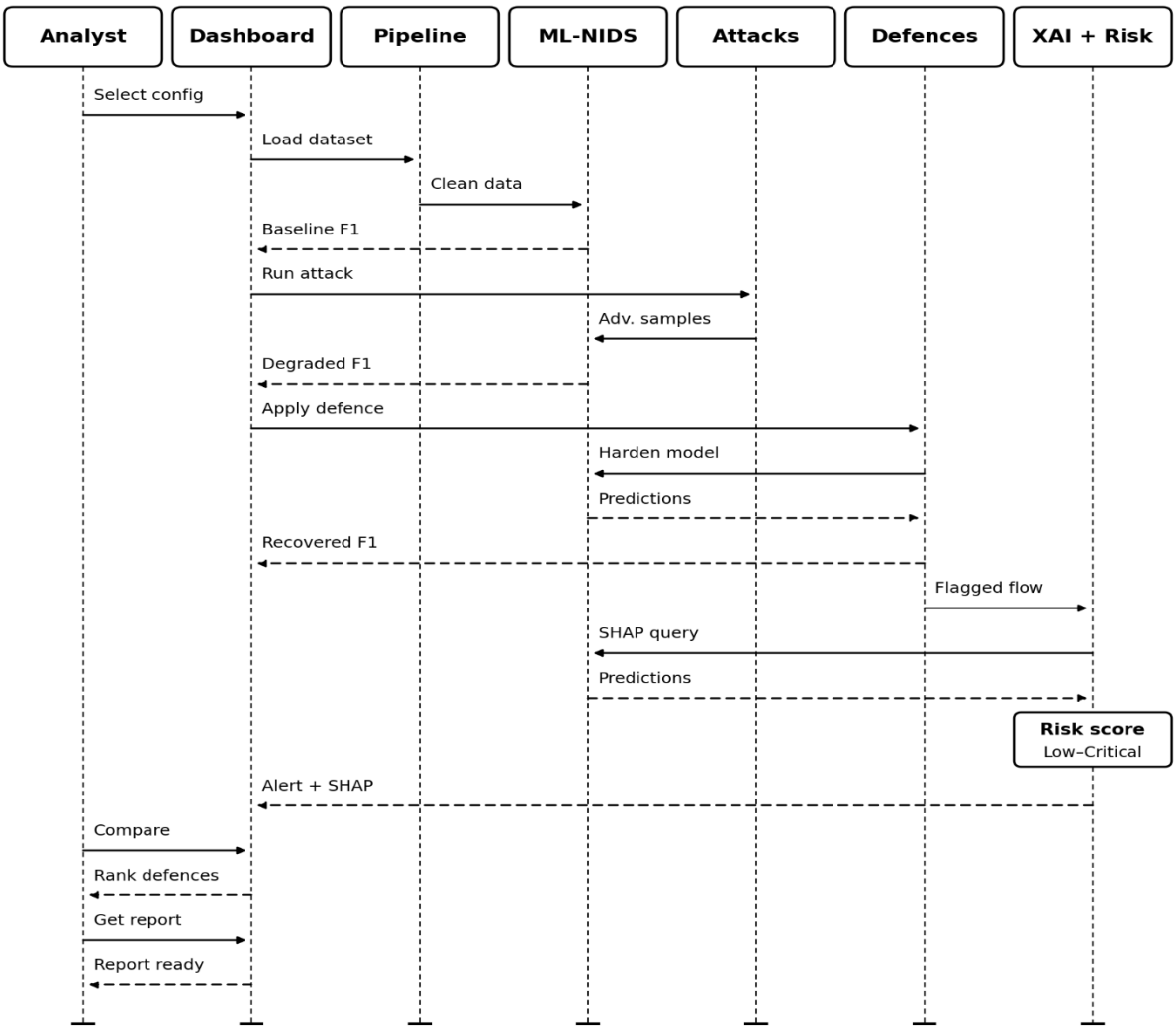


Figure 3.5: Sequence Diagram

3.7 Threat Model and Attack Surface

The diagram maps each attack surface in AdversaGuard to the threat it faces, the attack simulated against it, and the defence applied. Training data is exposed to poisoning (label-flipping and backdoor), which is countered by data sanitisation. Inference input is exposed to evasion (FGSM and PGD), which is countered by feature squeezing and autoencoder-based runtime detection. The trained model is exposed to degraded detection from all simulated attacks, which is countered by adversarial training and the combined defence.

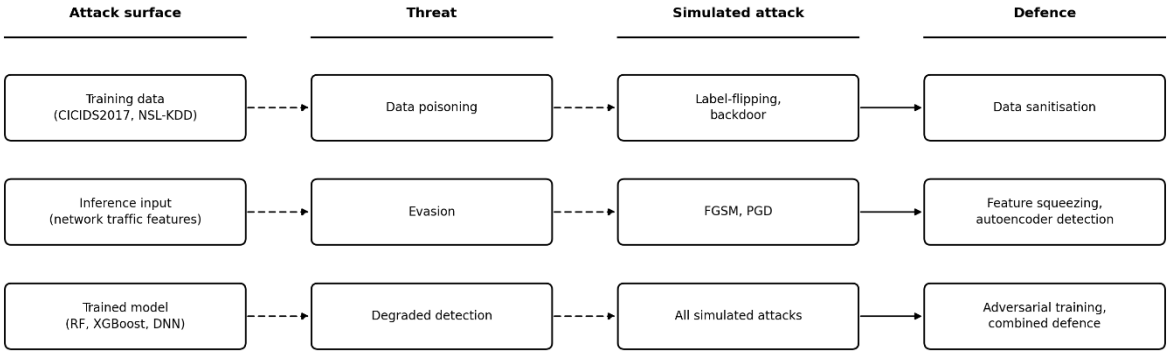


Figure 3.6: AdversaGuard Attack Surface and Threat Model

3.8 Security Controls

The system uses the following security controls:

- Dataset integrity checking
- Input validation
- Model integrity protection
- Training data sanitisation
- Runtime anomaly detection
- SHAP-based explanation
- Risk assessment
- Security alerts
- Dashboard access control
- Secure report handling

CHAPTER 4

SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The data collection stage is responsible for obtaining suitable network traffic data for developing and evaluating the intrusion detection models. **CICIDS2017** is used as the primary dataset because it contains both benign and different types of malicious network traffic. The dataset provides various network traffic features that can be used to identify abnormal and malicious activities.

The framework can also use **UNSW-NB15** as an additional dataset for evaluation. Using more than one dataset helps in analysing the behaviour of the models under different network traffic conditions. The collected data is divided into training and testing portions. The training data is used to build the machine learning models, while the testing data is used to evaluate their performance.

The collected datasets also provide the input required for adversarial attack experiments. The same prepared data can be used to study both clean and manipulated network traffic, allowing the system to compare normal model performance with performance under adversarial conditions.

4.2 DATA PREPROCESSING

Data preprocessing is an important stage because raw network traffic data may contain missing values, unnecessary features, categorical values, and inconsistent data formats. The collected datasets are therefore cleaned and transformed before being provided to the machine learning models.

The preprocessing process includes **removing or handling invalid and missing values, selecting relevant features, converting categorical information into suitable numerical representations, and applying feature scaling where required**. The target labels are also prepared so that the models can distinguish between normal and malicious network traffic.

After preprocessing, the data is converted into a suitable format for **Random Forest, XGBoost, and DNN** models. The same preprocessing procedure is maintained during testing so that the models receive consistent input. Proper preprocessing helps reduce unwanted data variations and supports reliable model training and evaluation.

4.2.1 ALGORITHMS / METHODOLOGY

The proposed AdversaGuard methodology follows a sequence of stages to evaluate and improve the security of ML-based NIDS.

1. **Dataset Collection:** Network traffic data is collected from CICIDS2017 and, where applicable, UNSW-NB15.

2. **Data Preprocessing:** The collected data is cleaned, transformed, and prepared for machine learning.

3. **ML Model Development:** Random Forest, XGBoost, and DNN models are trained to classify network traffic as normal or malicious.

4. **Clean Data Evaluation:** The trained models are first evaluated using clean testing data to establish their normal detection performance.

5. **Adversarial Attack Simulation:** Evasion attacks such as **FGSM and PGD** are used to generate manipulated inputs. Training-time attacks such as **label flipping and backdoor attacks** are used to study the effect of poisoned data.

6. **Defence Application:** Defence techniques such as **data sanitisation, adversarial training, feature squeezing, and autoencoder-based runtime detection** are applied to improve model robustness.

7. **Detection and Analysis:** The system analyses predictions using **SHAP-based Explainable AI**, risk scores, and security alerts.

8. **Defence Comparison:** The performance of different defence techniques is compared to identify their effectiveness under different attack conditions.

9. **Dashboard and Reporting:** The Streamlit dashboard displays attack results, model performance, risk information, and defence comparisons. Automated security reports are generated for further analysis.

4.2.2 Implementation Technology

The main technologies used for implementation are:

Table 4.1: Implementation Technology

COMPONENT	TECHNOLOGY
Programming Language	Python
Dashboard	Streamlit
Data Processing	Pandas, NumPy
ML Models	Scikit-learn, XGBoost
Deep Learning	TensorFlow/Keras or PyTorch
Explainable AI	Shap
Visualization	Matplotlib
Dataset	CICIDS2017, NSL-KDD
Development	VS Code / Jupyter Notebook

4.2.3 RESULTS AND DISCUSSION

The results and discussion stage evaluates how the selected machine learning models perform under both **normal and adversarial conditions**. Random Forest, XGBoost, and DNN models are evaluated using clean network traffic first and are then tested against different evasion and poisoning attacks.

The comparison helps identify the weaknesses of each model when exposed to adversarial manipulation. The defence techniques are then applied to determine whether the models show improved robustness after protection. The results can be compared using suitable performance measures such as **accuracy, precision, recall, F1-score, detection performance, and robustness under attack**, depending on the experiments performed.

The **SHAP-based Explainable AI module** helps explain the important features influencing the model's predictions. The risk scoring and alert system provides a simple indication of the security severity of detected activities. The defence comparison module helps identify which defence approach performs better for a particular attack scenario.

Finally, the results are presented through the **Streamlit dashboard**, where the security analyst can view model predictions, attack outcomes, defence performance, explanations, risk scores, and alerts. The system also generates an **automated security report** containing the important experimental findings.

CHAPTER 5

CONCLUSION

AdversaGuard is an integrated framework designed to evaluate and defend Machine Learning-based Network Intrusion Detection Systems against adversarial attacks. It evaluates Random Forest, XGBoost and DNN models on CICIDS2017, with NSL-KDD as a complementary dataset. Under clean-data evaluation, the models achieved accuracies of 98.21%, 98.64% and 97.83% and F1-scores of 97.94%, 98.37% and 97.51%, which formed the baseline. FGSM and PGD evasion attacks reduced the F1-score by approximately 12–18% and 17–23%, while label-flipping and backdoor poisoning attacks reduced it by approximately 8.6% and 11.4%. To improve robustness, the framework applies data sanitisation, adversarial training, feature squeezing, autoencoder-based runtime detection and a combined defence. The combined defence performed best, achieving an F1-score of approximately 96.84% compared with 78.92% for the undefended attacked model. SHAP-based explainability identifies the features behind each prediction, with an average analysis time of about 0.84 seconds per batch. A configurable 0–100 risk score classified the 10,000 evaluated events into Low (5,240), Medium (2,610), High (1,470) and Critical (680) categories. The Streamlit dashboard brings these components together, covering dataset selection, attack simulation, defence comparison, risk assessment and alerts, and it generates an automated security report in approximately 2.3 seconds. Overall, AdversaGuard provides a controlled environment for studying the effect of adversarial attacks on ML-based NIDS and for comparing defence mechanisms.

REFERENCES

1. I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, Funchal, Portugal, 2018, pp. 108–116.

2. M. Tavallaee, E. Bagheri, W. Lu and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proc. IEEE Symp. Computational Intelligence for Security and Defense Applications (CISDA)*, Ottawa, ON, Canada, 2009, pp. 1–6, doi: 10.1109/CISDA.2009.5356528.

3. I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, “A detailed analysis of the CICIDS2017 dataset,” Canadian Institute for Cybersecurity, University of New Brunswick, Fredericton, NB, Canada, 2017. [Online].

4. I. Goodfellow, J. Shlens and C. Szegedy, “Explaining and harnessing adversarial examples,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

5. A. Madry, A. Makelov, L. Schmidt, D. Tsipras and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” in *Proc. Int. Conf. Learning Representations (ICLR)*, Vancouver, BC, Canada, 2018.

6. A. Vassilev, A. Oprea, A. Fordyce and H. Anderson, “Adversarial machine learning: A taxonomy and terminology of attacks and mitigations,” National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST AI 100-2, Jan. 2024, doi: 10.6028/NIST.AI.100-2e2023.

7. A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies and M. Hamin, “Adversarial machine learning: A taxonomy and terminology of attacks and mitigations,” National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST AI 100-2e2025, Mar. 2025, doi: 10.6028/NIST.AI.100-2e2025.

8. T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

9. L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.

10. S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, Long Beach, CA, USA, 2017, pp. 4765–4774.

11. S. M. Lundberg, G. G. Erion and S.-I. Lee, “Consistent individualized feature attribution for tree ensembles,” *arXiv preprint arXiv:1802.03888*, 2018.

12. N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik and A. Swami, “The limitations of deep learning in adversarial settings,” in *Proc. IEEE European Symp. Security and Privacy (EuroS&P)*, Saarbrücken, Germany, 2016, pp. 372–387, doi: 10.1109/EuroSP.2016.36.

13. N. Papernot, P. McDaniel, A. Sinha and M. Wellman, “SoK: Security and privacy in machine learning,” in *Proc. IEEE European Symp. Security and Privacy (EuroS&P)*, London, U.K., 2018, pp. 399–414, doi: 10.1109/EuroSP.2018.00035.